

DESeq2_biotech

Sara González Bodí y Laura Rodríguez Casillas (Few modifications by Jaime Salama & Alberto Romero)

22 junio, 2025

Contents

Important web tutorials/manuals	1
Installation packages	2
Get the paths for all the directories and read the files needed	2
ONLY RUN FOR HTSeq-counts OUTPUTS	2
Upload data for HTSeq-count users and create DESeq object (NOT RUN)	2
ONLY RUN FOR FeaturesCounts OUTPUTS	3
Upload data for HTSeq-count users and create DESeq object	3
Prefiltering options before running DESeq2	4
Run the Differential Expression Analysis	6
Comparison: run DESeq2 for the comparison -> altered vs control	11
Create DESeq2 dataset object	11
Useful plots for Differential Expression Analysis	12
Gene Ontology enrichment annotation with ClusterProfiler	14
#Few modifications made by Jaime Salama & Alberto Romero	

Important web tutorials/manuals

1. DESeq2: <https://bioconductor.org/packages/devel/bioc/vignettes/DESeq2/inst/doc/DESeq2.html>
2. Gene Ontology with ClusterProfiler: <https://yulab-smu.top/biomedical-knowledge-mining-book/clusterprofiler-go.html>

Installation packages

```
## All packages loaded successfully.
```

Get the paths for all the directories and read the files needed

```
# Directory where the counts files are stored
# In case you work with htseq counts output
# counts_directory <- ""
# In case you work with feature counts output
counts_directory <- "../05-counts_alignment/"
# Output directory
output_dirertory <- "./output_DESeq2"
# Table with metadata (you have to create it by your own)
fullsamples <- read.csv("./metadata.csv")

# Finally, if the output directory does not exist, create it
if (!dir.exists(output_dirertory)){ dir.create(output_dirertory, recursive = TRUE)}
```

ONLY RUN FOR HTSeq-counts OUTPUTS

Upload data for HTSeq-count users and create DESeq object (NOT RUN)

```
# Get the samples files sorted and their names
sampleFiles <- sort(list.files(counts_directory, pattern = "*.counts"))

# Get the samples names
get.sample.name <- function(fname) strsplit(basename(fname), ".counts")[[1]][1]
sample.names <- unname(sapply(sampleFiles, get.sample.name))

# Create a table with the samples and files names
sampleTable <- data.frame(sampleName = sample.names,
                          fileName = sampleFiles)

# Merge the samples table with the samples information
merge <- merge(sampleTable, fullsamples,
               by.x="sampleName", by.y = "SampleID")

## Create the DESeq dataset object from HTSeq count files
ddsHTSeq <- DESeqDataSetFromHTSeqCount(sampleTable = merge,
                                       directory = counts_directory, design = ~Condition)
```

ONLY RUN FOR FeaturesCounts OUTPUTS

Upload data for HTSeq-count users and create DESeq object

```
# Get the samples files sorted and their names
sampleFiles <- sort(list.files(counts_directory, pattern = "*.txt$"))

# Get the samples names
get.sample.name <- function(fname) strsplit(basename(fname), ".txt")[[1]][1]
sample.names <- fullsamples[[1]]

# Create a table with the samples and files names
sampleTable <- data.frame(sampleName = sample.names,
                          fileName = sampleFiles)

# Merge the samples table with the samples information
merge <- merge(sampleTable, fullsamples,
               by.x="sampleName", by.y = "SampleID")

# Order the merged table by sampleName
merge <- merge[order(merge$sampleName, decreasing = TRUE),]

# Initialize an empty variable to store counts for each sample
countData <- NULL

# Loop through each sample file and read the featureCounts files
#for (i in 1:length(sampleFiles)) {
# Get the complete path to the featureCounts file we are iterating over
countFile <- file.path(counts_directory, sampleFiles)
# Read the featureCounts output file
countTable <- read.delim(countFile, header = TRUE, comment.char="#", row.names = 1)
# Extract the correct count column (usually the last column, but this can vary)
countColumn <- countTable[, 6:ncol(countTable)]
# Add the count column to the countData matrix, preserving row names (gene names)
#if (is.null(countData)) {
countData <- countColumn
#} else {
#countData <- cbind(countData, countColumn)
#}
#}

# Set the row names of countData to the gene names (from the first file, assumed to be the same across
rownames(countData) <- rownames(countTable)

# Set the column names for the countData matrix to the sample names
colnames(countData) <- sample.names

# Make sure the merge DataFrame matches the order of columns in countData
merge <- merge[match(colnames(countData), merge$sampleName), ]

#rownames will be the exact same as the columns names (the samples)
rownames(merge) <- colnames(countData)
```

LOOP

#Column 6

```

# Check if the dimensions match
if (ncol(countData) == nrow(merge)) {
  print("Sample count matches!")
} else {
  stop("Number of samples does not match between countData and merge")
}

```

```
## [1] "Sample count matches!"
```

```

# Ensure that 'Condition' is a factor (it should already be, but let's ensure)
merge$Condition <- factor(merge$Condition)

# Create the DESeqDataSet
ddsHTSeq <- DESeqDataSetFromMatrix(countData = countData, colData = merge,
                                   design = ~Condition)

# Check the DESeqDataSet object
ddsHTSeq

```

```

## class: DESeqDataSet
## dim: 23164 6
## metadata(1): version
## assays(1): counts
## rownames(23164): ENSRNOA050005711 ENSRNOA050005713 ... Vradi2697s00010
##      ENSRNOA050005864
## rowData names(0):
## colnames(6): SRR12506729 SRR12506736 ... SRR12506734 SRR12506735
## colData names(3): sampleName fileName Condition

```

Prefiltering options before running DESeq2

```

# Perform a minimal pre-filtering to keep only rows that have at least 10 (decided below) reads total

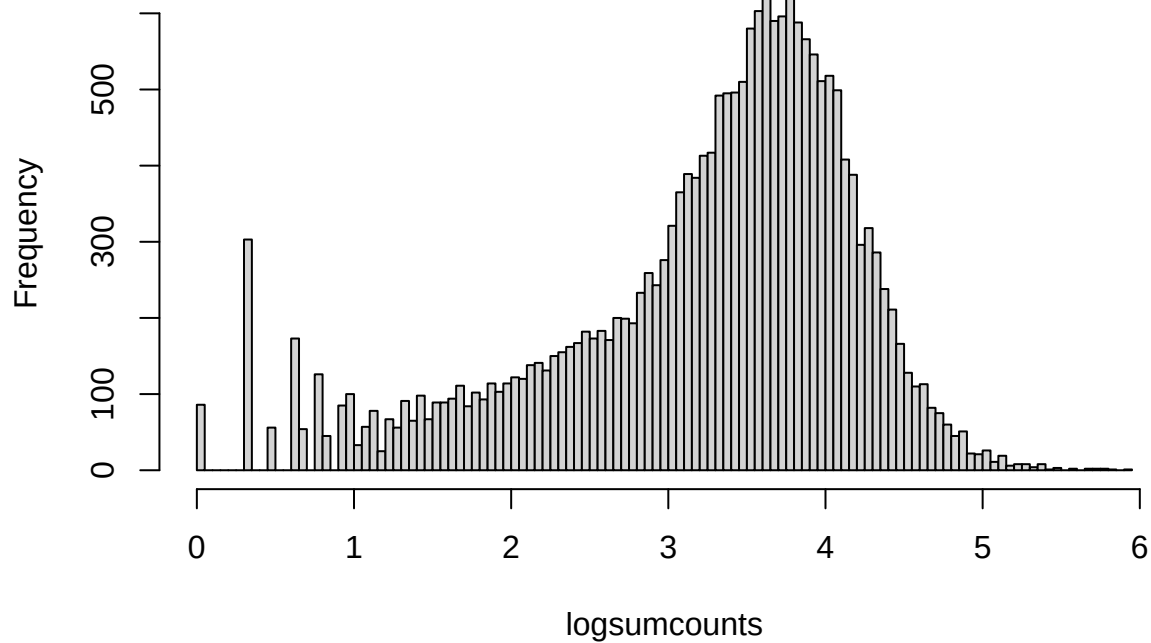
# Sum count for each gene across samples
sumcounts <- rowSums(counts(ddsHTSeq))

# take the log of the sum for each gene
logsumcounts <- log(sumcounts, base=10)

# 1st plot a histogram of the log scaled counts
# This will be useful in order to decide in which value keep the genes
plot_without_filter <- hist(logsumcounts, breaks=100, main="Histogram of the log scaled counts")

```

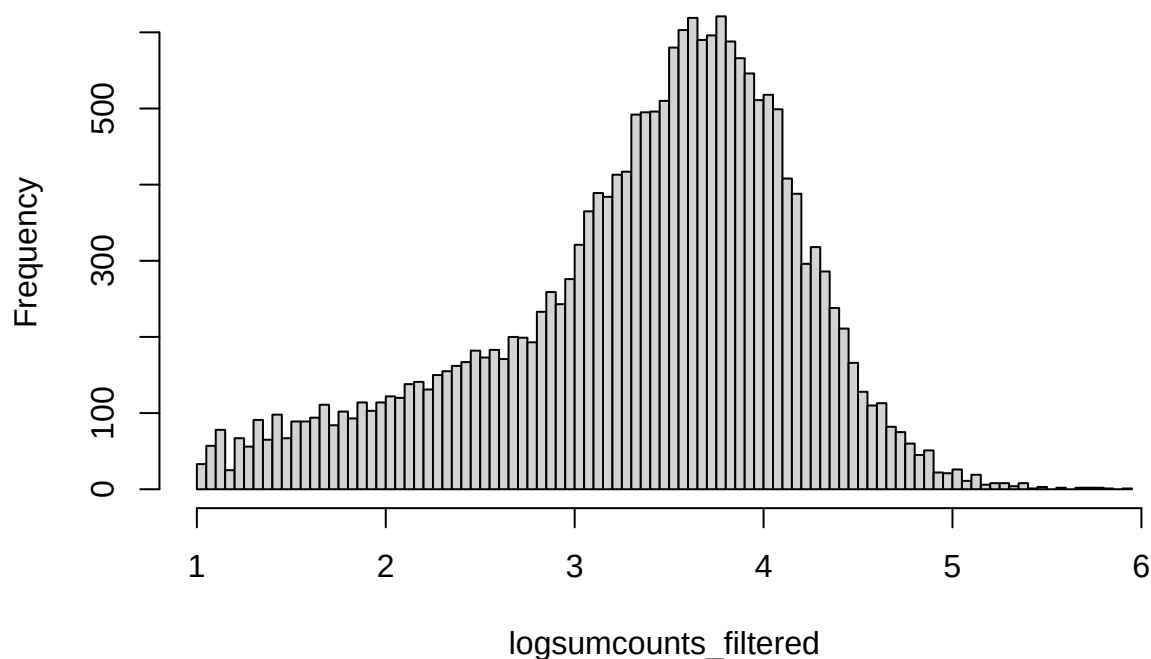
Histogram of the log scaled counts



```
# Get the genes with summed counts greater than 10! (This value could vary anytime you run another RNA-seq)
# This remove the lowly expressed genes
keep <- sumcounts > 10
# Keep the genes which sumcounts > 10 from the ddsHTSeq object
ddsHTSeq_filter <- ddsHTSeq[keep,]

# Finally, do another histogram but this time with the filtered ddsHTSeq object
sumcounts_filtered <- rowSums(counts(ddsHTSeq_filter))
logsumcounts_filtered <- log(sumcounts_filtered,base=10)
plot_with_filter <- hist(logsumcounts_filtered,breaks=100,main="Histogram of the log scaled counts \n keep > 10")
```

Histogram of the log scaled counts keep only the genes with summed counts greater than 50



Run the Differential Expression Analysis

```
# Run the Differential Expression Analysis
dds <- DESeq(ddsHTSeq_filter)
```

```
## estimating size factors
```

```
## estimating dispersions
```

```
## gene-wise dispersion estimates
```

```
## mean-dispersion relationship
```

```
## final dispersion estimates
```

```
## fitting model and testing
```

```
# Print the names of the results in the Differential Expression Analysis
print(resultsNames(dds))
```

```
## [1] "Intercept"
```

```
"Condition_Inoculated_vs_control"
```

```

# IMPORTANT: the following data transformation is only done for REPRESENTATION,
# it is not meant to be used for analysis
## Normalize the counts
normalized_counts <- counts(dds, normalized=TRUE)

## Log transform the normalized counts - Get the Median Absolute Deviation per gene (1 means by row)
normalized_counts_mad <- apply(normalized_counts, 1, mad)
## Order the normalized counts matrix using the Median Absolute Deviation
normalized_counts <- normalized_counts[order(normalized_counts_mad, decreasing=T), ]

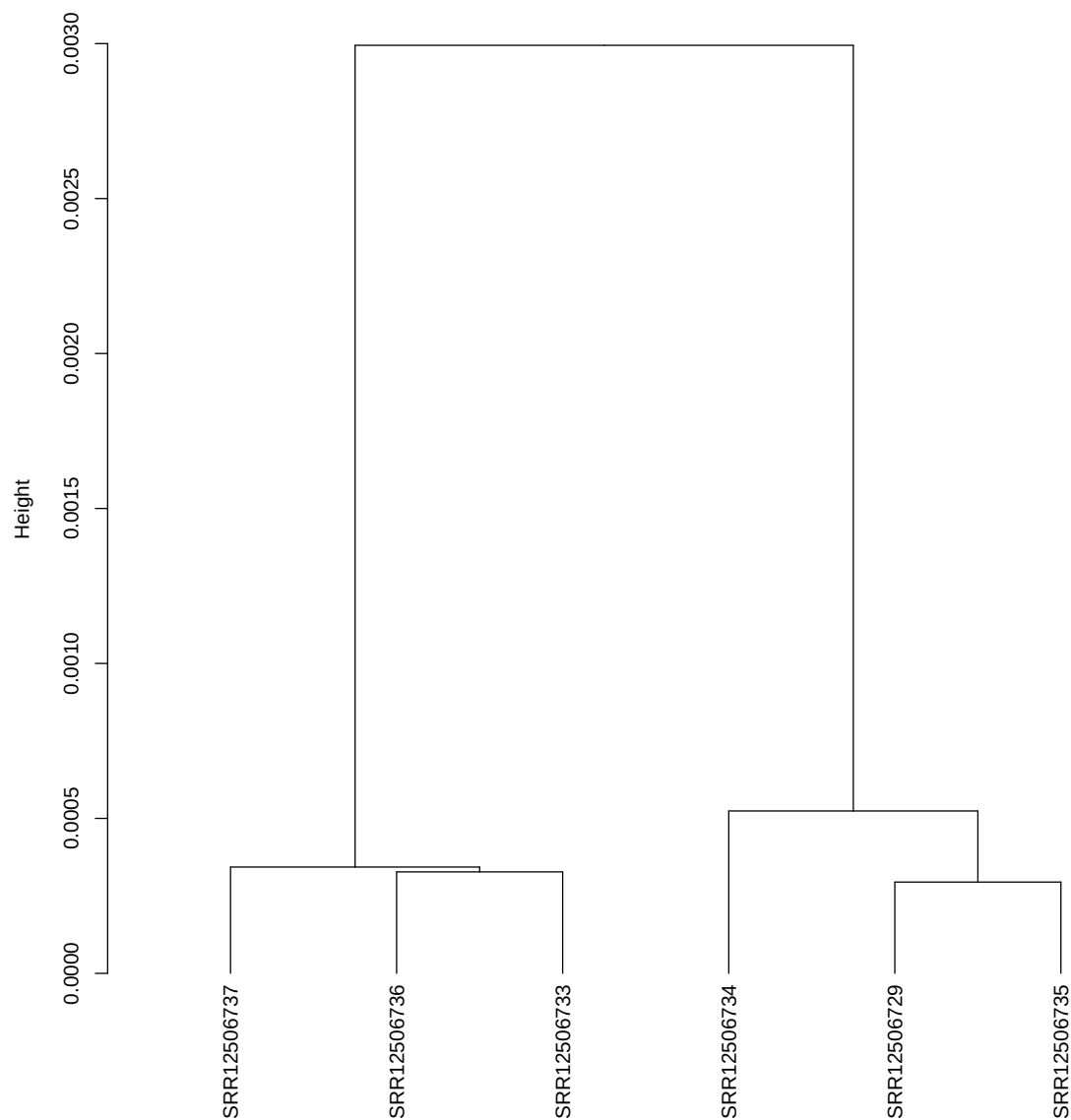
## Another method to create a variance.stabilized transformed counts for visualization
vsd <- vst(dds, blind=FALSE)

rlogMat <- assay(vsd)
rlogMat <- rlogMat[order(normalized_counts_mad, decreasing=T), ]

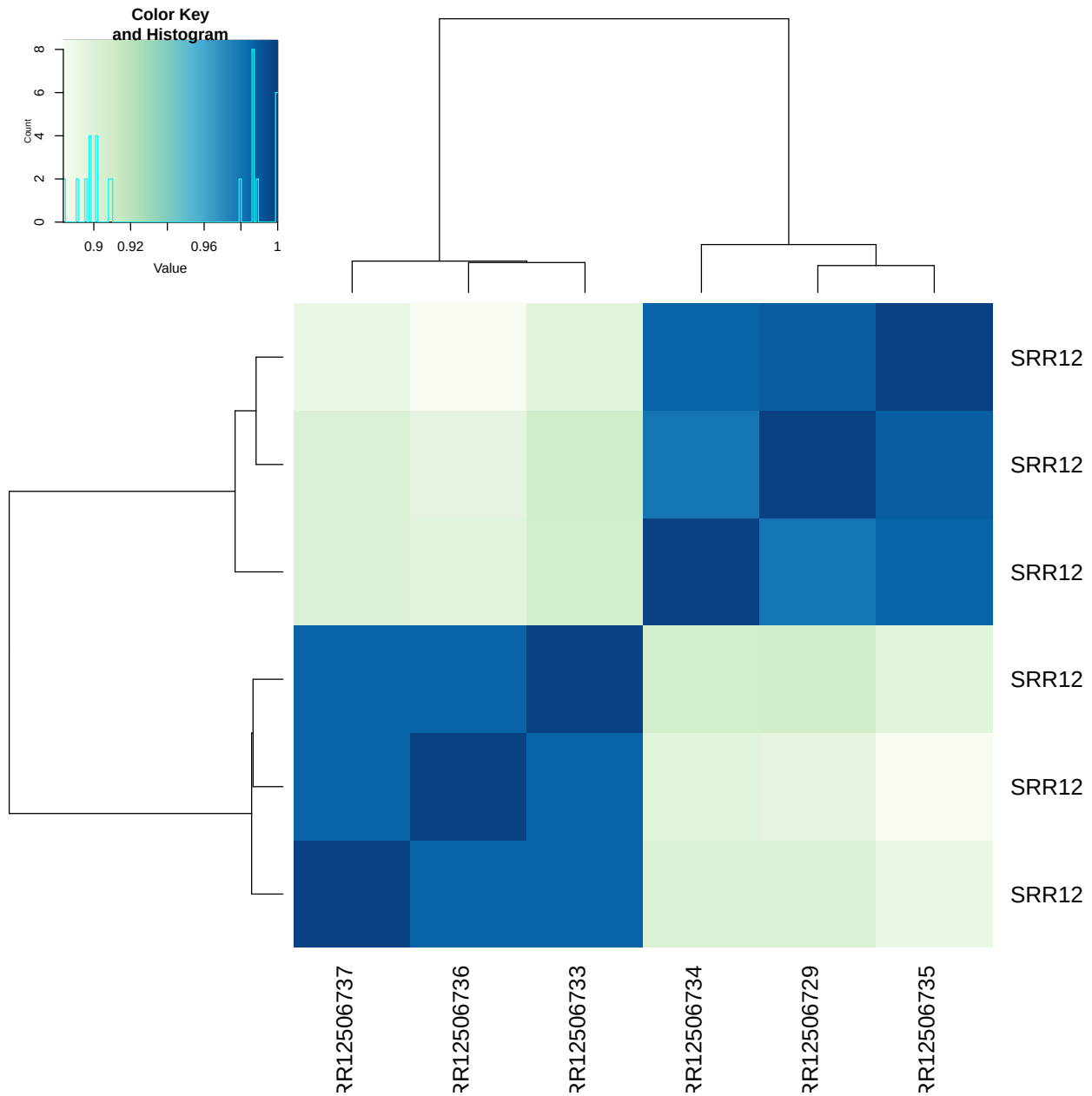
## Hierarchical clustering
hc <- hcluster(t(rlogMat), method="pearson")
# Get a dendrogram to check for outliers
TreeC = as.dendrogram(hc, method="average")
plot(TreeC, main = "Sample Clustering-checking outliers ", ylab = "Height")

```

Sample Clustering-checking outliers



```
# Estimate the pearson correlation to see it as a dendrogram!
pearson_cor <- as.matrix(cor(rlogMat, method="pearson"))
hmcol <- colorRampPalette(brewer.pal(9, "GnBu"))(100) # This is to establish the color palette
heatmap.2(pearson_cor, Rowv=as.dendrogram(hc), symm=T, trace="none",
          col=hmcol, margins=c(8,5))
```

```
# plot PCA
plotPCA(vsd, intgroup="Condition") + ggtitle("PCA Plot of Variance Stabilizing Transformation") +
  geom_point(size = 1) +
  theme(axis.line = element_line(colour = "black"),
        plot.title = element_text(angle = 0, size = 14, face = 'bold', vjust = 0.5, hjust = 0.5),
        plot.subtitle = element_blank(), plot.caption = element_blank(), # subtitle elements

        axis.text.x = element_text(angle = 0, size = 12, vjust = 1), # x axis title
        axis.text.y = element_text(angle = 0, size = 12, vjust = 0.5), # y axis title
        axis.title = element_text(size = 13), # both axis titles

        legend.position = 'right', legend.key.size = unit(0.5, 'cm'), # legend position
        legend.text = element_text(size = 12), legend.title = element_blank()) + # legend text
  theme_bw() +
```

```

theme(
  panel.grid.minor = element_blank(),
  panel.grid.major = element_blank(),
  panel.border = element_rect(size = 0.20),
  axis.title.x = element_text(size=11),
  axis.title.y = element_text(size = 11, margin = margin(t = 0, r = 10, b = 0, l = 0)),
  axis.text.x = element_text(color="black",size=10),
  axis.text.y = element_text(color="black",size=10),
  legend.position = "right") +
theme(plot.title = element_text(hjust = 0.5))

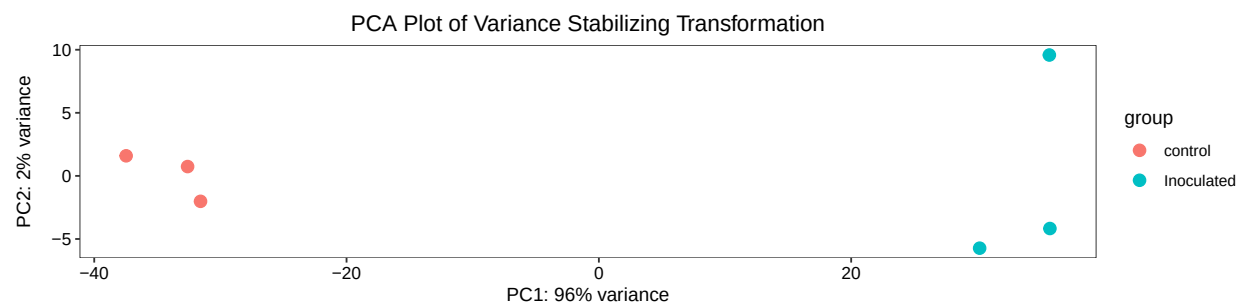
```

```
## using ntop=500 top features by variance
```

```

## Warning: The `size` argument of `element_rect()` is deprecated as of ggplot2 3.4.0.
## i Please use the `linewidth` argument instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.

```



Comparison: run DESeq2 for the comparison $\rightarrow$ altered vs control

Create DESeq2 dataset object

```
# What sample is the reference in our analysis?
ref <- "control"

# Relabel the filtered DESeq2 object by the reference in the specific analysis
ddsHTSeq_filter$Condition <- relevel(ddsHTSeq_filter$Condition, ref = ref)
```

```
## Normalize and calculate dispersion with the reference of the specific analysis
dds <- DESeq(ddsHTSeq_filter) # <- normalize and calculate dispersion
```

```
## estimating size factors
```

```
## estimating dispersions
```

```
## gene-wise dispersion estimates
```

```
## mean-dispersion relationship
```

```
## final dispersion estimates
```

```
## fitting model and testing
```

```
print (resultsNames(dds))
```

```
## [1] "Intercept" "Condition_Inoculated_vs_control"
```

```
# Get the comparison results including the condition
paired_comparison_results <- results(dds, name="Condition_Inoculated_vs_control", pAdjustMethod="BH")
# Custom the output comparison
paired_comparison_results_tb <- paired_comparison_results %>% data.frame() %>%
rownames_to_column(var="gene") %>% as_tibble()

# Store the output comparison data frame to a csv file
# Create the name for the output folder
comparison_name_folder <- "Condition_Inoculated_vs_control"
# Get the complete path to store the output and create the directory if it does not already exists
output_path <- file.path(output_dir, comparison_name_folder)
if (!dir.exists(output_path)) {dir.create(file.path(output_path), recursive = TRUE)}

# Applying normalization to data (ignoring design) - for clustering, etc.
dataf <- vst(dds, blind = TRUE) %>% assay()
dataf <- dataf %>% as_tibble(rownames = "gene")

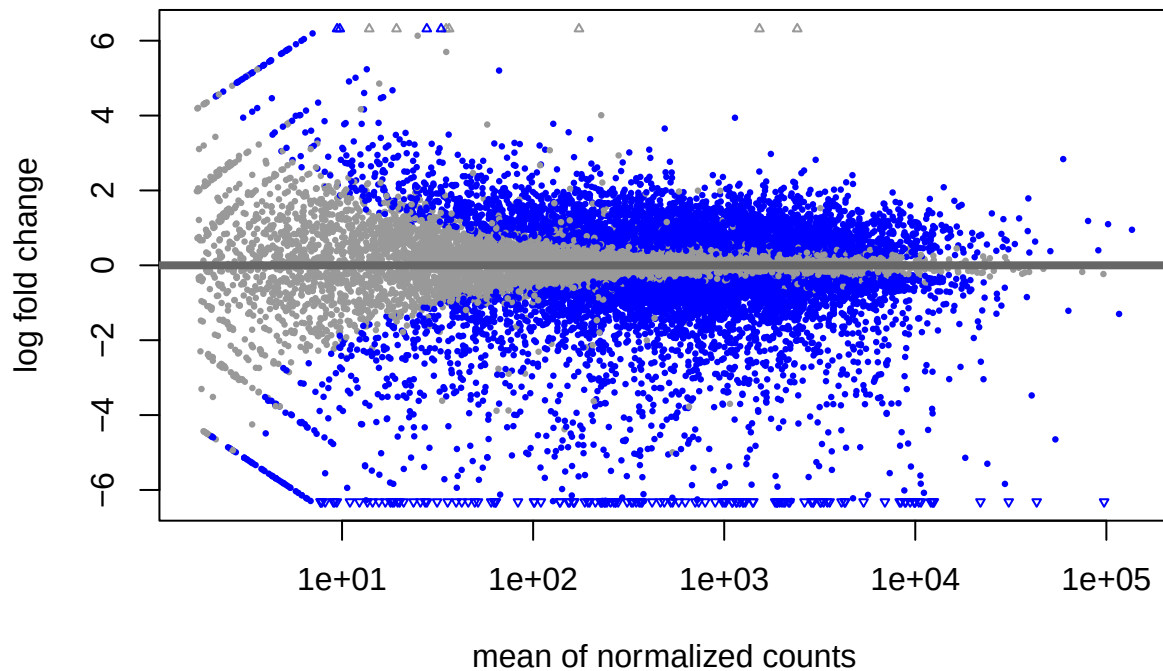
# Set the thresholds where you want to filter your data
padj.cutoff <- 0.01
lfc.cutoff <- 1 # remember log!

# Merge the output table from the comparison with the counts (dataf)
# Add also another column that includes TRUE/FALSE in case the gene pass the thresholds set
resdata <- merge(as.data.frame(paired_comparison_results_tb), as.data.frame(dataf), by = 'gene', sort =

# Write the complete table to the output folder
write.table(resdata, file = paste(output_path, (paste0(comparison_name_folder, "_final_results.csv")),
```

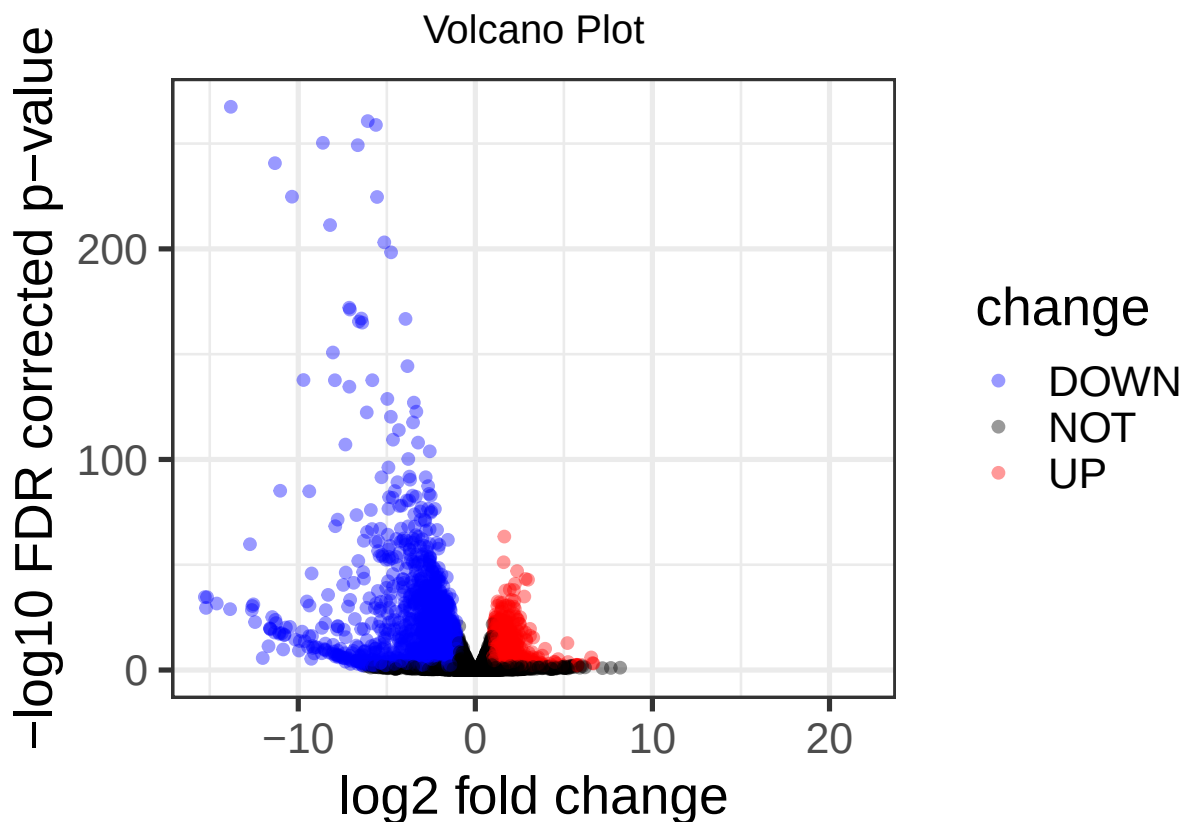
Useful plots for Differential Expression Analysis

```
# Create MA plot
plotMA(dds)
```



```
# Create a volcano plot
# Add a variable to resdata to include UP/DOWN or NOT taking into account the thresholds
resdata$change <- as.factor(ifelse(
  is.na(resdata$padj) | resdata$padj > padj.cutoff,
  'NOT',
  ifelse(resdata$padj <= padj.cutoff & abs(resdata$log2FoldChange) > lfc.cutoff,
    ifelse(resdata$log2FoldChange > lfc.cutoff, 'UP', 'DOWN'),
    'NOT')))
# Create the plot
volcanoPlot = ggplot(data=resdata, aes(x=log2FoldChange, y=-log10(padj), color=change)) +
  geom_point(alpha=0.4, size=1.75) +
  theme_set(theme_set(theme_bw(base_size=20)))+
  xlab("log2 fold change") + ylab("-log10 FDR corrected p-value") +
  ggtitle("Volcano Plot" ) + theme(plot.title = element_text(size=15,hjust = 0.5))+
  scale_colour_manual(values = c('blue','black','red')) ## corresponding to the levels(res
# This is to show the plot
show(volcanoPlot)
```

```
## Warning: Removed 25 rows containing missing values or values outside the scale range
## (`geom_point()`).
```



Gene Ontology enrichment annotation with ClusterProfiler

```
# We need to get the Universe of genes or background genes
background_genes <- as.vector(resdata$gene)

# We need to get the significant genes in the DEGs (Differentially Expressed Genes)
interesting_set <- resdata %>% filter (abs(log2FoldChange) >= lfc.cutoff & padj <= padj.cutoff)
interesting_set <- as.vector(interesting_set$gene)

#Enrichment will be done using Vigna Radiata genome without having *.db object.

#Reading data from ensembl BioMart and filtering for null rows
ensembl_data <- read.csv("./mart_export.csv")

term2gene <- data.frame(
  TERM_ID = ensembl_data$GO.term.accession,
  Gene = ensembl_data$Gene.stable.ID
)
term2gene_filtered <- term2gene[!(is.na(term2gene[,1]) | term2gene[,1] == ""), ]

term2name <- data.frame(
  TERM_ID = ensembl_data$GO.term.accession,
  name = paste(ensembl_data$GO.term.name, ensembl_data$GO.domain, sep = ";")
)
```

```

    )
term2name_filtered <- term2name[!(is.na(term2name[,1]) | term2name[,1] == ""), ]

# Run the Gene Ontology Enrichment analysis
enrichment_custom <- enricher( interesting_set, # DEGs list
                                TERM2GENE = term2gene_filtered, # A data.frame with the first column containing th
                                TERM2NAME = term2name_filtered, # A data.frame with the first column of term ID an
                                universe = background_genes, # the backgroud genes
                                pvalueCutoff = 0.05, # Adjusted pvalue Cutoff on enrichment test to report
                                pAdjustMethod = "BH", # Method for the p.adjusted to be calculated
                                qvalueCutoff = 0.05)

# Get the result from the enrichment as a data frame
ora_analysis_bp_df <- as.data.frame(enrichment_custom@result)

# Get the representation of the Gene Ontology
barplot <- barplot(enrichment_custom,
                   showCategory = 100,
                   font.size = 8) +
  ggtitle("GO - Ontology ")

show(barplot)

```

